Aplicaciones moviles con JetPack Compose y Kotlin

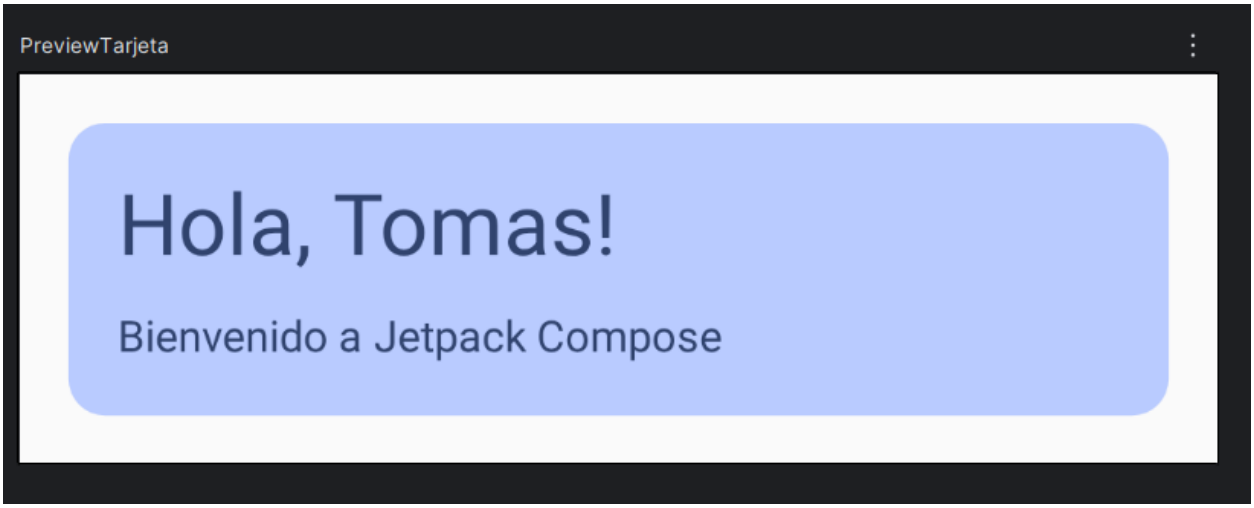
PROGRAMACION MOVIL 2026

Tomas Villegas Jaramillo

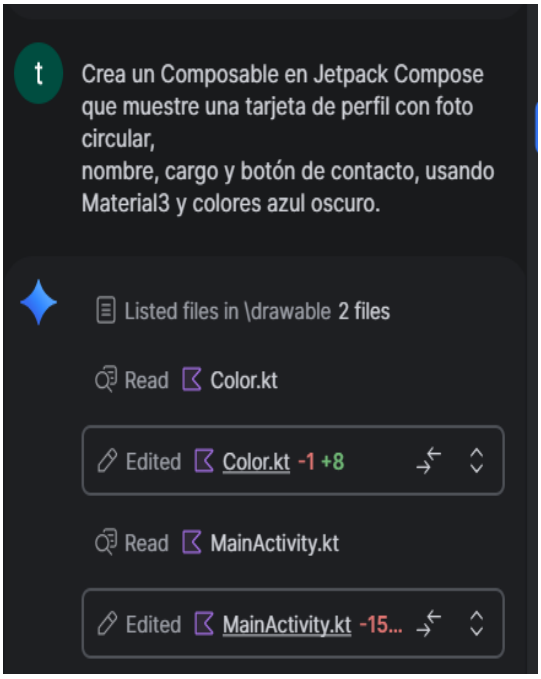
1) Primera APP

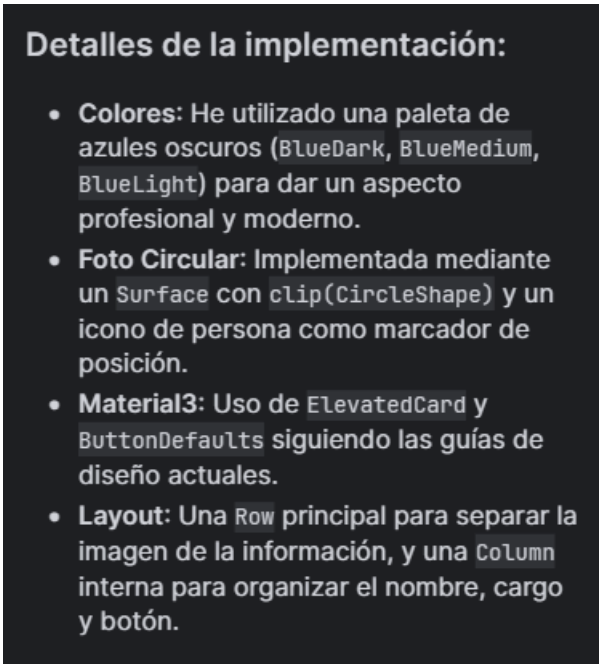
HOLA COMPOSE: TU PRIMERA PANTALLA DECLARATIVA
Nivel: Principiante · Duración estimada: 2 – 3 horas
Configura tu entorno, entiende la filosofía declarativa y construye tu primera UI sin XML.

- ✓ Preview tarjeta @composable, MainActvity.kt:

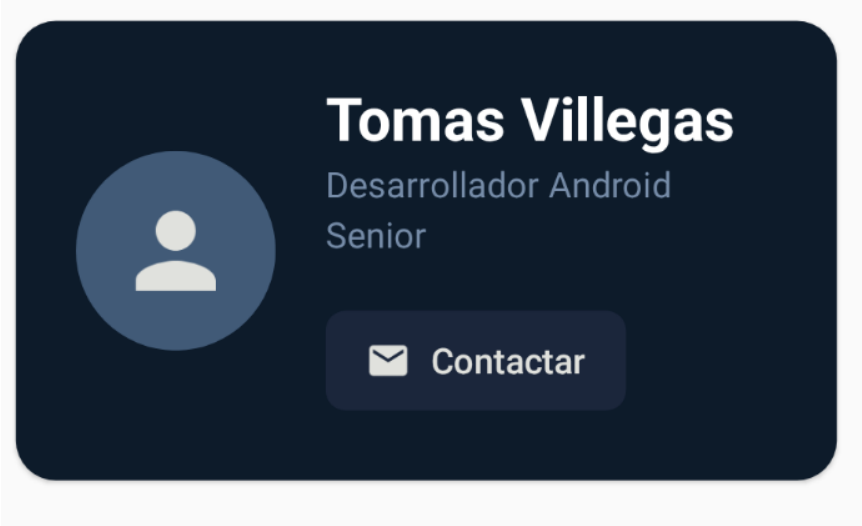


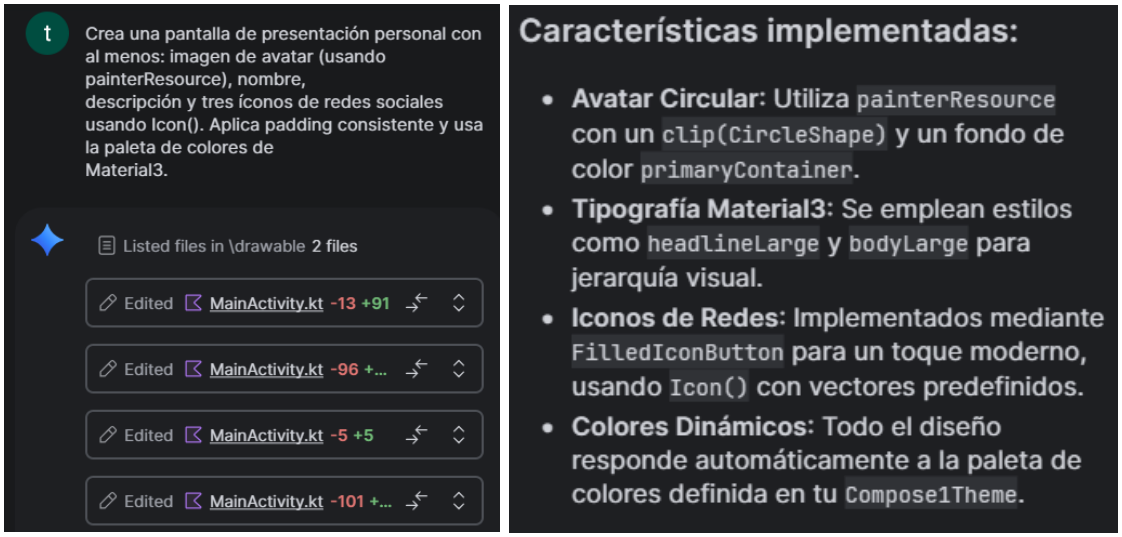
- ✓ Actividad con IA — Generación de UI
  Describe en Claude.ai o ChatGPT la siguiente pantalla:
  Crea un Composable en Jetpack Compose que muestre una tarjeta de perfil con foto circular,
  nombre, cargo y botón de contacto, usando Material3 y colores azul oscuro.
  Analiza el código generado, identifica los Composables usados.
  Modifícalo para personalizarlo con tus propios datos.
  Reflexión: ¿Qué partes debiste corregir? ¿Por qué la IA puede equivocarse?





- ✓ Despues de probar el promt la IA logro con el objetivo de crear un tarjeta y además usar los colores de material3. Tuve que corregir la forma en la que se visualizaba la tarjeta además cambie los datos genéricos por los mios.





2) Segunda APP

ESTADO Y NAVEGACIÓN: APP DE LISTA DE TAREAS
Nivel: Básico-Intermedio · Duración estimada: 3 – 4 horas
Maneja estado reactivo con remember/mutableStateOf y crea navegación entre pantallas con NavController.

✓ Añadir dependencia de Navigation Compose en build.gradle.

Build.gradle.kts (app:module):

```
implementation(libs.androidx.navigation.compose)
```

Libs.versions.toml:

```
androidx-navigation-compose = { group = "androidx.navigation", name = "navigation-compose", version.ref = "navigation" }
```

- ✓ Modelar la data class Tarea(id, titulo, completada).

```
data class Tarea(
    val id: Int,
    val titulo: String,
    val completada: Boolean = false
)
```

- ✓ Crear la pantalla ListaTareasScreen con LazyColumn.

```
LazyColumn(
    modifier = Modifier
        .fillMaxSize()
        .padding( paddingValues = innerPadding)
        .padding(horizontal = 16.dp)
) {
```

- ✓ Implementar el diálogo de nueva tarea con AlertDialog y TextField.

```
// Tarea con AlertDialog y TextField
if (mostrarDialogo) {
    NuevaTareaDialog(
        onDismiss = { mostrarDialogo = false },
        onConfirm = { titulo ->
            val nuevoId = (tareas.maxOfOrNull { it.id } ?: 0) + 1
            onTareasUpdate(tareas + Tarea(nuevoId, titulo))
            mostrarDialogo = false
        }
    )
}
```

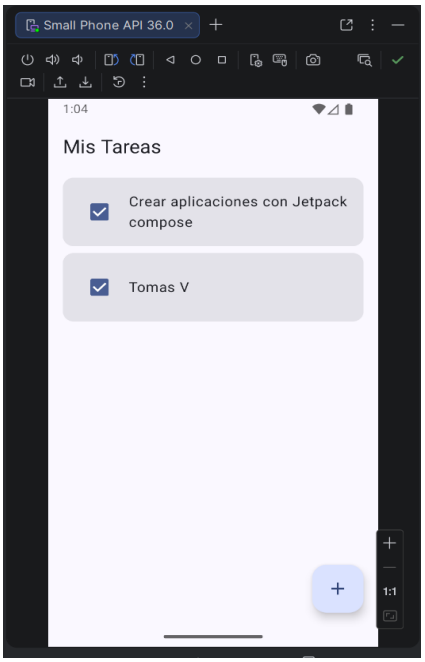
✓ Configurar NavHost con dos rutas: lista y detalle.

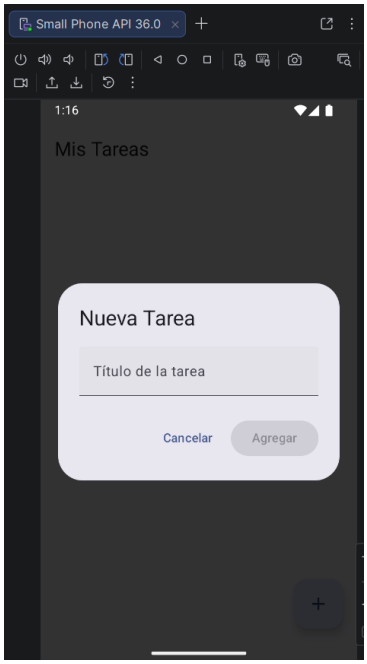
```
fun AppNavigation() {
    val navController = rememberNavController()
    // Estado de las tareas en un nivel superior para persistencia simple durante la sesión
    var tareas by remember { mutableStateOf( value = listOf<Tarea>()) }

    // Configurar NavHost con dos rutas: lista y detalle
    NavHost(navController = navController, startDestination = "lista") {
        composable( route = "lista") {
            ListaTareasScreen(
                tareas = tareas,
                onTareasUpdate = { tareas = it },
                onNavigateToDetail = { id ->
                    // 12. Pasar el id de tarea como argumento de navegación
                    navController.navigate( route = "detalle/$id")
                }
            )
        }
        composable(
            route = "detalle/{tareaId}",
            arguments = listOf(navArgument( name = "tareaId") { type = NavType.IntType })
        ) { backStackEntry ->
            val id = backStackEntry.arguments?.getInt( key = "tareaId") ?: 0
            val tarea = tareas.find { it.id == id }
            DetalleTareaScreen(tarea = tarea, onBack = { navController.popBackStack() })
        }
    }
}
```

✓ Pasar el id de tarea como argumento de navegación.

```
items(tareas, key = { it.id }) { tarea ->
    TareaItem(
        tarea = tarea,
        onCheckedChange = { checked ->
            onTareasUpdate(tareas.map {
                if (it.id == tarea.id) it.copy(completada = checked) else it
            })
        },
        onDelete = {
            onTareasUpdate(tareas.filter { it.id != tarea.id })
        },
        onClick = { onNavigateToDetail(tarea.id) }
    )
}
```





✓ Actividad con IA — Componentes de Lista

Prompt para Copilot / Claude:

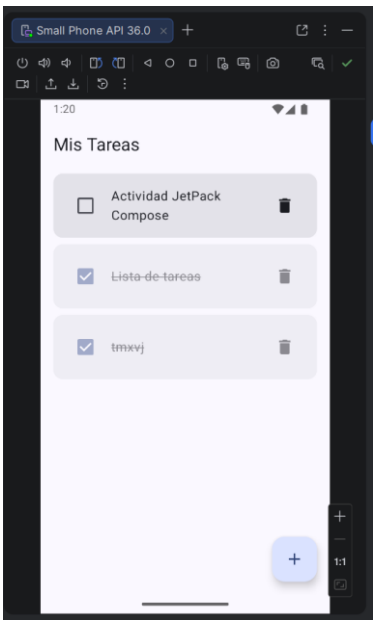
Crea un Composable TareaItem en Jetpack Compose con Checkbox, texto tachado si está completada,

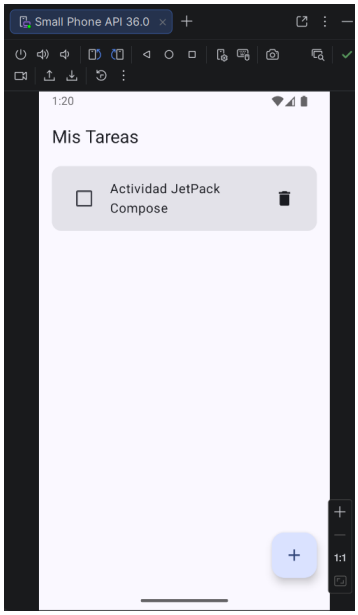
ícono de eliminar a la derecha y animación fade al marcar como hecha. Material3.

Prueba el código generado, ajusta colores y comportamiento.

Añade la funcionalidad de swipe-to-delete usando SwipeToDismiss.

Documenta qué aprendiste del código de la IA que no conocías.





3) Tercera APP

ESTADO Y NAVEGACIÓN: APP DE LISTA DE TAREAS
Nivel: Básico-Intermedio · Duración estimada: 3 – 4 horas
Maneja estado reactivo con remember/mutableStateOf y crea navegación entre pantallas con NavController.

✓ **Configurar Retrofit + Gson/Moshi en el módulo de datos.**

Libs.versions.toml:

```
retrofit = "3.0.0"
```

```
retrofit = { group = "com.squareup.retrofit2", name = "retrofit", version.ref = "retrofit" }
retrofit-converter-gson = { group = "com.squareup.retrofit2", name = "converter-gson", version.ref = "retrofit" }
```

Build.gradle.kts:

```
// Retrofit + Gson
implementation(libs.retrofit)
implementation(libs.retrofit.converter.gson)
```

✓ **Crear WeatherRepository con función suspendida getWeather(city).**

```
// Crear WeatherRepository con función suspendida getWeather(city)
2 Usages
class WeatherRepository {
    1 Usage
    suspend fun getWeather(city: String): WeatherData {
        delay( timeMillis = 1000) // Simulación de carga de red
        return WeatherData(city, temperature = "22°C", description = "Nublado")
    }
}
```

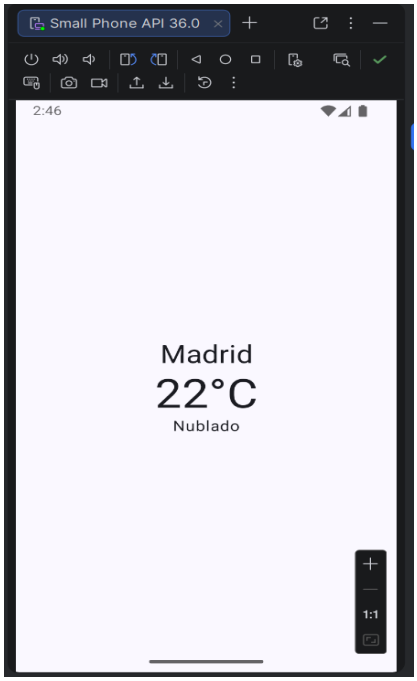
✓ **Definir WeatherUiState como sealed class (Loading, Success, Error).**

```
// Definir WeatherUiState como sealed class (Loading, Success, Error)
12 Usages   3 Inheritors
sealed class WeatherUiState {
    3 Usages
    object Loading : WeatherUiState()
    2 Usages
    data class Success(val data: WeatherData) : WeatherUiState()
    2 Usages
    data class Error(val message: String) : WeatherUiState()
}
```

✓ **Implementar WeatherViewModel que expone StateFlow<WeatherUiState>.**

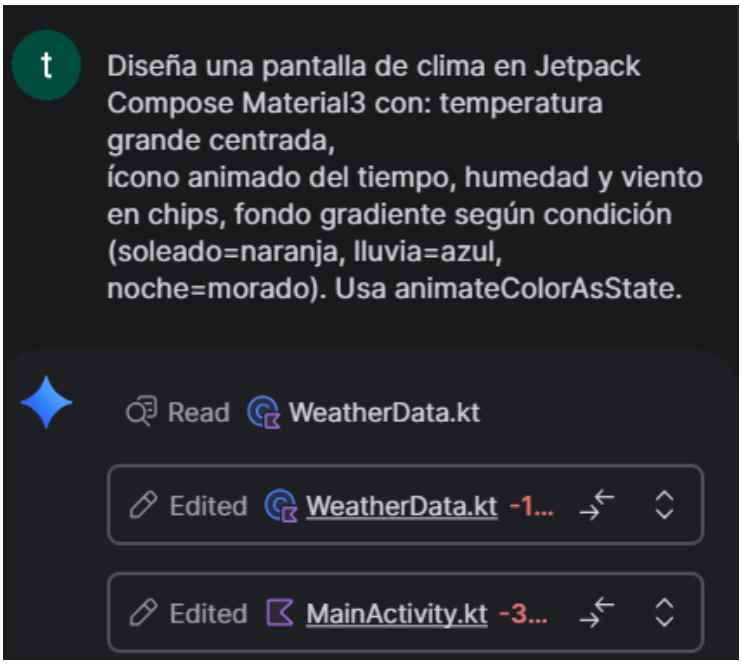
```
//Implementar WeatherViewModel que expone StateFlow<WeatherUiState>
1 Usage
class WeatherViewModel(private val repository: WeatherRepository = WeatherRepository()) : ViewModel() {
    4 Usages
    private val _uiState = MutableStateFlow<WeatherUiState>( value = WeatherUiState.Loading)
    1 Usage
    val uiState: StateFlow<WeatherUiState> = _uiState.asStateFlow()

    1 Usage
    fun loadWeather(city: String) {
        viewModelScope.launch {
            _uiState.value = WeatherUiState.Loading
            try {
                val data = repository.getWeather(city)
                _uiState.value = WeatherUiState.Success(data)
            } catch (e: Exception) {
                _uiState.value = WeatherUiState.Error(e.message ?: "Error desconocido")
            }
        }
    }
}
```



- ✓ **Actividad con IA — Diseño de la Pantalla del Clima**
  Diseña una pantalla de clima en Jetpack Compose Material3 con: temperatura grande centrada, ícono animado del tiempo, humedad y viento en chips, fondo gradiente según condición (soleado=naranja, lluvia=azul, noche=morado). Usa animateColorAsState.

  Evaluacion: ¿es usable? ¿accesible? ¿sigue Material You?:
  El diseño es altamente usable gracias a una jerarquía clara donde la temperatura y el contexto visual (gradientes) dominan, aunque los chips inferiores podrían quedar ocultos en pantallas pequeñas. En cuanto a accesibilidad, presenta riesgos críticos de contraste al usar texto blanco sobre fondos brillantes (naranja/amarillo) y carece de descripciones de voz (contentDescription) para lectores de pantalla. Finalmente, aunque sigue la estética de Material You (M3) en formas y tipografía, prioriza la expresividad del clima sobre el Color Dinámico del sistema, lo cual es aceptable en apps meteorológicas pero rompe la cohesión global del dispositivo.



- ✓ Agrega un buscador con historial de ciudades usando Room Database. El ViewModel debe cargar el historial al iniciar y
  persistir cada búsqueda exitosa. Implementa un LazyRow de chips con las últimas 5 ciudades buscadas.

4) Cuarta APP

ROOM + HILT + CLEAN ARCHITECTURE: APP DE FINANZAS PERSONALES
Nivel: Intermedio-Avanzado · Duración estimada: 5 – 6 horas
Persistencia local con Room, inyección de dependencias con Hilt y arquitectura limpia en capas.

✓ Configurar Hilt: plugins hilt-android, hilt-compiler, anotación @HiltAndroidApp en Application.

```
@HiltAndroidApp
class FinanzasApp : Application()
```

```
// Hilt
implementation(libs.hilt.android)
kapt(libs.hilt.compiler)
implementation(libs.androidx.hilt.navigation.compose)
```

- ✓ Definir TransaccionEntity con @Entity, @PrimaryKey(autoGenerate=true).

```
import androidx.room.Entity
import androidx.room.PrimaryKey

25 Usages
@Entity(tableName = "transacciones")
data class TransaccionEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val descripcion: String,
    val monto: Double,
    val tipo: String,       // "INGRESO" o "GASTO"
    val categoria: String,
    val fecha: Long = System.currentTimeMillis()
)
```

- ✓ Crear TransaccionDao con @Query, @Insert, @Delete, Flow <List<Transaccion>>.

```
package com.example.compose4                                  1 11

import ...

4 Usages
@Dao
interface TransaccionDao {
    1 Usage
    @Query("SELECT * FROM transacciones ORDER BY fecha DESC")
    fun getAll(): Flow<List<TransaccionEntity>>

    1 Usage
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertar(t: TransaccionEntity)

    1 Usage
    @Delete
    suspend fun eliminar(t: TransaccionEntity)

    @Query("SELECT SUM(monto) FROM transacciones WHERE tipo = :tipo")
    suspend fun getTotalPorTipo(tipo: String): Double?
}
```

- ✓ Implementar el Use Case AgregarTransaccionUseCase con validación de campos.

```
fun agregarTransaccion(descripcion: String, monto: Double, tipo: String, categoria: String) {
    viewModelScope.launch {
        repo.insertar(
            t = TransaccionEntity(
                descripcion = descripcion,
                monto = monto,
                tipo = tipo,
                categoria = categoria
            )
        )
    }
}
```

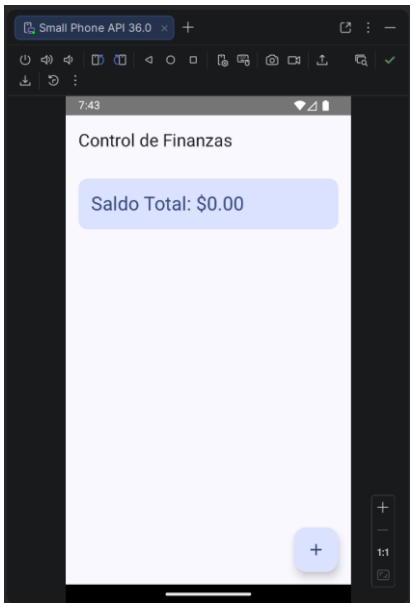
- ✓ Inyectar el repositorio en el ViewModel con @HiltViewModel e @Inject.

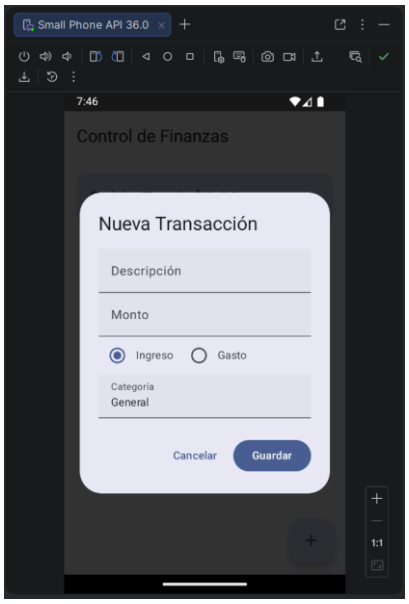
```
@HiltViewModel
class FinanzasViewModel @Inject constructor(
    private val repo: TransaccionRepository
) : ViewModel() {

    1 Usage
    val transacciones = repo.getAll().stateIn(
        scope = viewModelScope,
        started = SharingStarted.WhileSubscribed( stopTimeoutMillis = 5000),
        initialValue = emptyList()
    )

    1 Usage
    fun agregarTransaccion(descripcion: String, monto: Double, tipo: String, categoria: String) {
        viewModelScope.launch {
            repo.insertar(
                t = TransaccionEntity(
                    descripcion = descripcion,
                    monto = monto,
                    tipo = tipo,
                    categoria = categoria
                )
            )
        }
    }
```

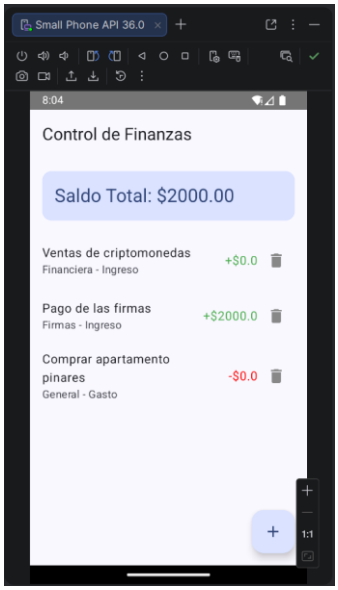
- ✓ Construir la pantalla de dashboard con resumen mensual y gráfica de barras.

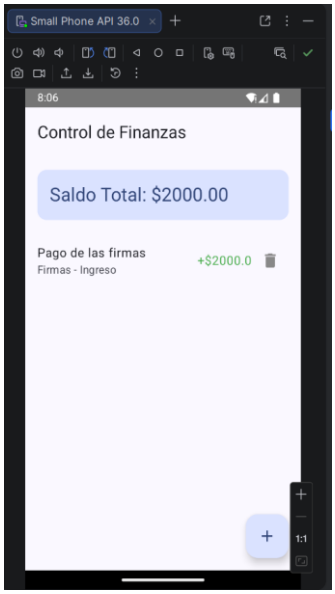




- ✓ Actividad con IA — Sistema de Diseño Completo
  Soy un estudiante de Android. Crea un sistema de diseño Material3 en Jetpack Compose para una app de finanzas personales: paleta de colores (claro/oscuro), tipografía, shapes, y 3 componentes reutilizables: TarjetaResumen, ChipCategoria y GraficoBarras. Implementa el Theme.kt generado y aplícalo a toda la app.

Herramienta extra: usa Figma + plugin Material Theme Builder para exportar tokens.





5) Quinta APP

APP COMPLETA CON IA INTEGRADA: ASISTENTE DE RECETAS INTELIGENTE
Nivel: Avanzado · Duración estimada: 6 – 8 horas

Proyecto integrador: cámara, reconocimiento de imágenes con ML Kit, sugerencias de recetas con API de IA y UX pulida.

✓ Configurar permisos de cámara en AndroidManifest y solicitarlos en runtime.

```
<uses-permission android:name="android.permission.CAMERA" />
<uses-feature android:name="android.hardware.camera" />
<uses-permission android:name="android.permission.INTERNET" />
```

✓ Implementar CameraX con PreviewView y captura de ImageProxy.

```
@Composable
fun CameraPreview(onIngredientsDetected: (List<String>) -> Unit) {
    val context = LocalContext.current
    val lifecycleOwner = LocalLifecycleOwner.current
    val cameraExecutor = remember { Executors.newSingleThreadExecutor() }
    val labeler = remember { ImageLabeling.getClient(ImageLabelerOptions.DEFAULT_OPTIONS) }

    AndroidView(factory = { ctx ->
        val previewView = PreviewView( context = ctx)
        val cameraProviderFuture = ProcessCameraProvider.getInstance( context = ctx)
        cameraProviderFuture.addListener({
            val cameraProvider = cameraProviderFuture.get()
            val preview = Preview.Builder().build().also { it.setSurfaceProvider(previewView.surfaceProvider) }
            val imageAnalysis = ImageAnalysis.Builder()
                .setBackpressureStrategy(ImageAnalysis.STRATEGY_KEEP_ONLY_LATEST)
                .build()
                .also { analysis ->
                    analysis.setAnalyzer(cameraExecutor) { imageProxy ->
                        val mediaImage = imageProxy.image
                        if (mediaImage != null) {...} else imageProxy.close()
                    }
                }
            cameraProvider.unbindAll()
            cameraProvider.bindToLifecycle(lifecycleOwner, CameraSelector.DEFAULT_BACK_CAMERA, ...useCases = preview, imageAnalysis)
        }, executor = ContextCompat.getMainExecutor( context = ctx))
        previewView
    }, modifier = Modifier.fillMaxSize())
}
```

✓ Hacer streaming de la respuesta y mostrarlo con efecto typewriter animado.

```
@Composable
fun TypewriterText(text: String) {
    var displayedText by remember { mutableStateOf( value = "") }

    LaunchedEffect( key1 = text) {
        if (text.length > displayedText.length) {
            val nextPart = text.substring( startIndex = displayedText.length)
            nextPart.forEach { char ->
                displayedText += char
                delay( timeMillis = 15)
            }
        } else if (text.isEmpty()) {
            displayedText = ""
        }
    }

    Card(
        colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surfaceVariant),
        modifier = Modifier.fillMaxWidth().padding(vertical = 8.dp)
    ) {
        Text(displayedText, modifier = Modifier.padding( all = 16.dp), fontSize = 14.sp)
    }
}
```

✓ Guardar recetas favoritas en Room y mostrarlas en el inicio sin conexión.

```
item {
    Spacer(modifier = Modifier.height(24.dp))
    Text("Mis Favoritos (Offline)", style = MaterialTheme.typography.titleLarge, fontWeight = FontWeight.Bold)
}

items(favorites) { recipe ->
    Card(modifier = Modifier.fillMaxWidth().padding(vertical = 4.dp)) {
        Column(modifier = Modifier.padding( all = 16.dp)) {
            Text(recipe.nombre, fontWeight = FontWeight.Bold)
            Text("${recipe.tiempo_minutos} min | ${recipe.dificultad}", style = MaterialTheme.typography.bodySmall)
        }
    }
}
```

✓ Actividad con IA — Diseño UX Completo con Prototipado

Fase 1 — Prototipado visual:

• Usa Uizard.io o Galileo AI: describe la app en lenguaje natural y genera mockups.

• Exporta las pantallas y úsalas como referencia de diseño.

Fase 2 — Código de pantallas:

•Prompt: Implementa en Jetpack Compose la pantalla de streaming de recetas con efecto typewriter, fondo con blur, chips de tiempo y dificultad, botón de guardar animado.

Fase 3 — Reflexión académica (500 palabras):

• ¿Cómo cambió tu flujo de trabajo con la IA? ¿Qué limitaciones encontraste?

Automatización de tareas repetitivas: sugerencias de código, generación de layouts y corrección de errores comunes se volvieron más rápidas.

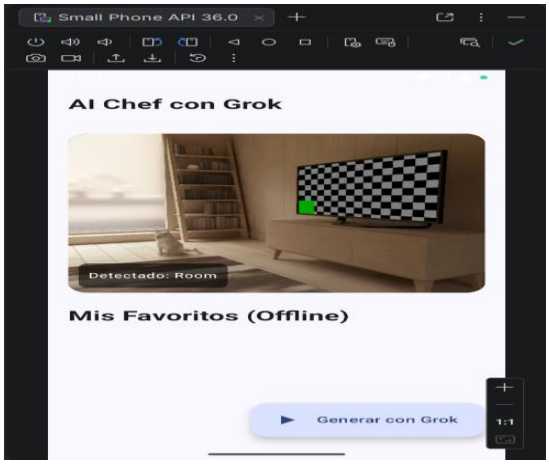
Menos tiempo en documentación: el agente propone snippets y explica APIs sin tener que salir de Android Studio.

Iteración más ágil: puedes probar ideas de UI/UX con prompts en lugar de escribir todo manualmente.

• ¿En qué aspectos fue útil y en cuáles fue imprecisa o peligrosa?

Dependencia del contexto: si el proyecto tiene configuraciones específicas (Gradle, librerías externas), el agente a veces no entiende bien y propone código incompatible.

Errores en proyectos grandes: en repositorios con muchas dependencias, las sugerencias pueden ser superficiales o no compilar.

Falta de criterio arquitectónico: no reemplaza buenas prácticas de diseño (MVVM, Clean Architecture), solo sugiere fragmentos.

Privacidad y seguridad: hay riesgo si se confía ciegamente en que la IA maneje datos sensibles o credenciales.